

Trabalho Prático 3 - APPOO

Nome: Guilherme Augusto Nunes

Matrícula: 2024017015

Código:

```
atividade_pratica_03_01.py ✕ ×
1  from abc import ABC, abstractmethod
2
3  class Jogador(ABC):
4      def __init__(self, nome: str):
5          if not nome.strip():
6              raise ValueError("O nome não pode ser vazio")
7          self.__nome = nome
8
9      @property
10     def nome(self): #getter
11         return self.__nome
12
13     @abstractmethod
14     def calcula_desempenho(self):
15         pass
16
17
18     class JogadorFutebol(Jogador):
19         def __init__(self, nome: str, gols: int, assistencias: int, partidas: int):
20             super().__init__(nome)
21             self.gols = gols
22             self.assistencias = assistencias
23             self.partidas = partidas
24
25         @property #getter de gols
26         def gols(self):
27             return self._gols
28
29         @property #getter de assist
30         def assistencias(self):
31             return self._assistencias
32
33         @property #getter de partidas
34         def partidas(self):
35             return self._partidas
36
37         @gols.setter
38         def gols(self, valor: int):
39             if valor < 0:
40                 raise ValueError("A quantidade de gols não pode ser negativa.")
41             self._gols = valor
42
```

```

42
43     @assistencias.setter
44     def assistencias(self, valor: int):
45         if valor < 0:
46             raise ValueError("A quantidade de assistencias não pode ser negativa.")
47         self._assistencias = valor
48
49     @partidas.setter
50     def partidas(self, valor: int):
51         if valor < 0:
52             raise ValueError("A quantidade de partidas não pode ser negativa.")
53         self._partidas = valor
54
55     def calcula_desempenho(self):
56         #cada gol é 15 pontos, assistencia 4 pontos, partidas 2 pontos
57         return (self._gols * 15) + (self._assistencias * 4) + (self._partidas * 2)
58
59
60 class JogadorBasquete(Jogador):
61     def __init__(self, nome: str, cestas: int, rebotes: int, assistencias: int):
62         super().__init__(nome)
63         self.cestas = cestas
64         self.rebotes = rebotes
65         self.assistencias = assistencias
66
67     @property #getter de cestas
68     def cestas(self):
69         return self._cestas
70
71     @property #getter de rebotes
72     def rebotes(self):
73         return self._rebotes
74
75     @property #getter de assistencias
76     def assistencias(self):
77         return self._assistencias
78
79     @cestas.setter
80     def cestas(self, valor: int):
81         if valor < 0:
82             raise ValueError("A quantidade de cestas não pode ser negativa.")
83         self._cestas = valor
84
85     @rebotes.setter
86     def rebotes(self, valor: int):
87         if valor < 0:
88             raise ValueError("A quantidade de rebotes não pode ser negativa.")
89         self._rebotes = valor
90
91     @assistencias.setter
92     def assistencias(self, valor: int):
93         if valor < 0:
94             raise ValueError("A quantidade de assistencias não pode ser negativa.")
95         self._assistencias = valor
96
97     def calcula_desempenho(self):
98         #cada cesta é 10 pontos, rebote 5 pontos, assistencia 3 pontos
99         return (self._cestas * 10) + (self._rebotes * 5) + (self._assistencias * 3)
100
101
102 class JogadorVolei(Jogador):
103     def __init__(self, nome: str, aces: int, bloqueios: int, ataques: int):
104         super().__init__(nome)
105         self.aces = aces
106         self.bloqueios = bloqueios
107         self.ataques = ataques
108
109     @property #getter de aces
110     def aces(self):
111         return self._aces
112
113     @property #getter de bloqueios
114     def bloqueios(self):
115         return self._bloqueios
116

```

```

116
117     @property #getter de ataques
118     def ataques(self):
119         return self._ataques
120
121     @aces.setter
122     def aces(self, valor: int):
123         if valor < 0:
124             raise ValueError("A quantidade de aces não pode ser negativa.")
125         self._aces = valor
126
127     @bloqueios.setter
128     def bloqueios(self, valor: int):
129         if valor < 0:
130             raise ValueError("A quantidade de bloqueios não pode ser negativa.")
131         self._bloqueios = valor
132
133     @ataques.setter
134     def ataques(self, valor: int):
135         if valor < 0:
136             raise ValueError("A quantidade de ataques não pode ser negativa.")
137         self._ataques = valor
138
139     def calcula_desempenho(self):
140         #cada ace é 12 pontos, bloqueio 5 pontos, ataque 7 pontos
141         return (self._aces * 12) + (self._bloqueios * 5) + (self._ataques * 7)
142
143
144 class TimeAnalise:
145     def __init__(self):
146         self._jogadores = []
147
148     def adiciona_jogadores(self, jogador: Jogador):
149         if not isinstance(jogador, Jogador):
150             raise TypeError("É necessário um objeto do tipo Jogador")
151         self._jogadores.append(jogador)
152         print(f"Jogador {jogador.nome} adicionado com sucesso")
153

```

```

153
154     def exibir_estatisticas(self):
155         if not self._jogadores:
156             print("Não existe jogadores cadastrados")
157             return
158
159         for j in self._jogadores:
160             desempenho = j.calcula_desempenho()
161             print(f"Jogador {j.nome}, desempenho: {desempenho}\n")
162
163     def melhor_desempenho(self): #identifica o jogador com melhor desempenho
164         if not self._jogadores:
165             print("Não existe jogadores cadastrados para realizar a análise")
166             return
167         melhor_jogador = self._jogadores[0]
168         maior_pontuacao = melhor_jogador.calcula_desempenho()
169
170         for j in self._jogadores[1:]:
171             pontuacao_atual = j.calcula_desempenho()
172             if pontuacao_atual > maior_pontuacao:
173                 maior_pontuacao = pontuacao_atual
174                 melhor_jogador = j
175
176         print(f"Destaque: {melhor_jogador.nome} com {maior_pontuacao} pontos")
177

```

```

178
179 if __name__ == "__main__":
180     analise = TimeAnalise()
181
182     try:
183         #instancias de tipos diferentes
184         jogador1 = JogadorFutebol("C. Ronaldo", 30, 10, 8)
185         jogador2 = JogadorBasquete("Michael Jordan", 20, 30, 15)
186         jogador3 = JogadorVolei("Giba", 20, 25, 28)
187
188         #erro
189         #jogador4 = JogadorFutebol("Maradona" -40)
190
191         analise.adiciona_jogadores(jogador1)
192         analise.adiciona_jogadores(jogador2)
193         analise.adiciona_jogadores(jogador3)
194         #analise.adiciona_jogadores(jogador4)
195
196     except ValueError as e:
197         print(f"Erro ao cadastrar: {e}")
198
199     print("\nEstatística dos Jogadores:\n")
200     analise.exibir_estatisticas()
201     analise.melhor_desempenho()

```

Saída no terminal:

```

C:\Users\Gui\AppData\Local\I X + v
Jogdor C. Ronaldo adicionado com sucesso
Jogdor Michael Jordan adicionado com sucesso
Jogdor Giba adicionado com sucesso

Estatística dos Jogadores:

Jogador C. Ronaldo, desempenho: 506

Jogador Michael Jordan, desempenho: 395

Jogador Giba, desempenho: 561

Destaque: Giba com 561 pontos
Press any key to continue . . . |

```

Exceção número negativo:

